

AI Usage Description

1. Scope Of AI Assistance

The application was not made by AI. The chess rules, C++ architecture, runtime model paths, visualization behavior, packaging, tests, and final integration were designed, implemented, checked, and owned by the students. AI assistants were used only as minor support tools during development.

AI was not used as a runtime component. The submitted program does not call AI services, online model APIs, or external agents while running.

AI assistance was limited mainly to:

- documentation wording and reorganization,
- debugging explanations for C++, CMake, raylib, and packaging issues,
- small review and checklist suggestions,
- comparing selected Java/TypeScript chess ideas with equivalent C++ designs,
- phrasing the AI-usage, summary, and user-facing manual text.

2. Human Review

AI output was treated as draft support material, not as authoritative project work. It was kept only after manual review, code inspection, and testing. Important checks included:

1. reading generated text or code before accepting it,
2. comparing chess logic against known chess rules and existing chess-domain experience,
3. running perft, unit, loader, tensor, evaluator, and MCTS tests,
4. checking actual raylib behavior in the live application,
5. removing or rewriting suggestions that did not match the architecture.

3. Areas Where AI Was Useful

AI was useful for repetitive or explanatory work:

- restructuring long Markdown documents,
- finding clearer names for sections and test descriptions,
- explaining compiler/build errors,
- pointing out stale documentation after code changes,
- generating draft wording for the AI usage, summary, and manual documents.

AI also helped compare ideas from prior chess projects with this C++ version, but the final C++ source was adapted to this repository's ownership, headers, tests, build system, and raylib UI.

4. Areas Where AI Needed Correction

AI output was often wrong or incomplete in project-specific details:

- Chess-rule edge cases around castling, en-passant, check, promotion, and undo state needed tests and manual verification.
- Neural-network dimensions, policy sizes, activation keys, and BT4 block/head counts had to be checked against the actual C++ implementation.
- UI layout suggestions were often too generic for a dense desktop visualizer and had to be redesigned manually.
- Documentation drafts sometimes preserved old assumptions after the code had changed, especially around startup configuration and BT4 model loading.

5. Student-Written And Reviewed Work

The core project work was implemented and reviewed by the students:

- C++ class structure across `src/chess`, `src/game`, `src/io`, `src/nn`, `src/search`, and `src/viz`,
- legal move generation, make/unmake, FEN/SAN, status helpers, and perft tests,
- linked-list move history,
- file I/O for config, FEN, PGN export support, model blobs, and test data,
- tensor operations and activation snapshots,
- NNUE, CNN, BT4, Classical, and MCTS implementation paths,
- raylib board rendering, controls, editor, dialogs, activation views, and tree view,
- build/test scripts, tutorial media, Windows package, and final documentation.

6. Final Position

AI was helpful for documentation, debugging explanations, and review support, but it did not make the application. The project required manual knowledge of chess rules, C++ ownership, binary formats, raylib rendering, tensor shapes, search behavior, packaging, and testing. The final deliverable was checked against the real code, the local model files, and the automated test suite.